

Class Activity-4

1. Study Hours vs Marks (R)

```
# Create dataset
```

```
StudyHours <- c(2,3,4,5,6,7,8,9)
```

```
Marks <- c(45,52,58,63,69,74,81,88)
```

```
data <- data.frame(StudyHours, Marks)
```

```
# Scatterplot
```

```
plot(data$StudyHours, data$Marks,
```

```
main="Study Hours vs Marks",
```

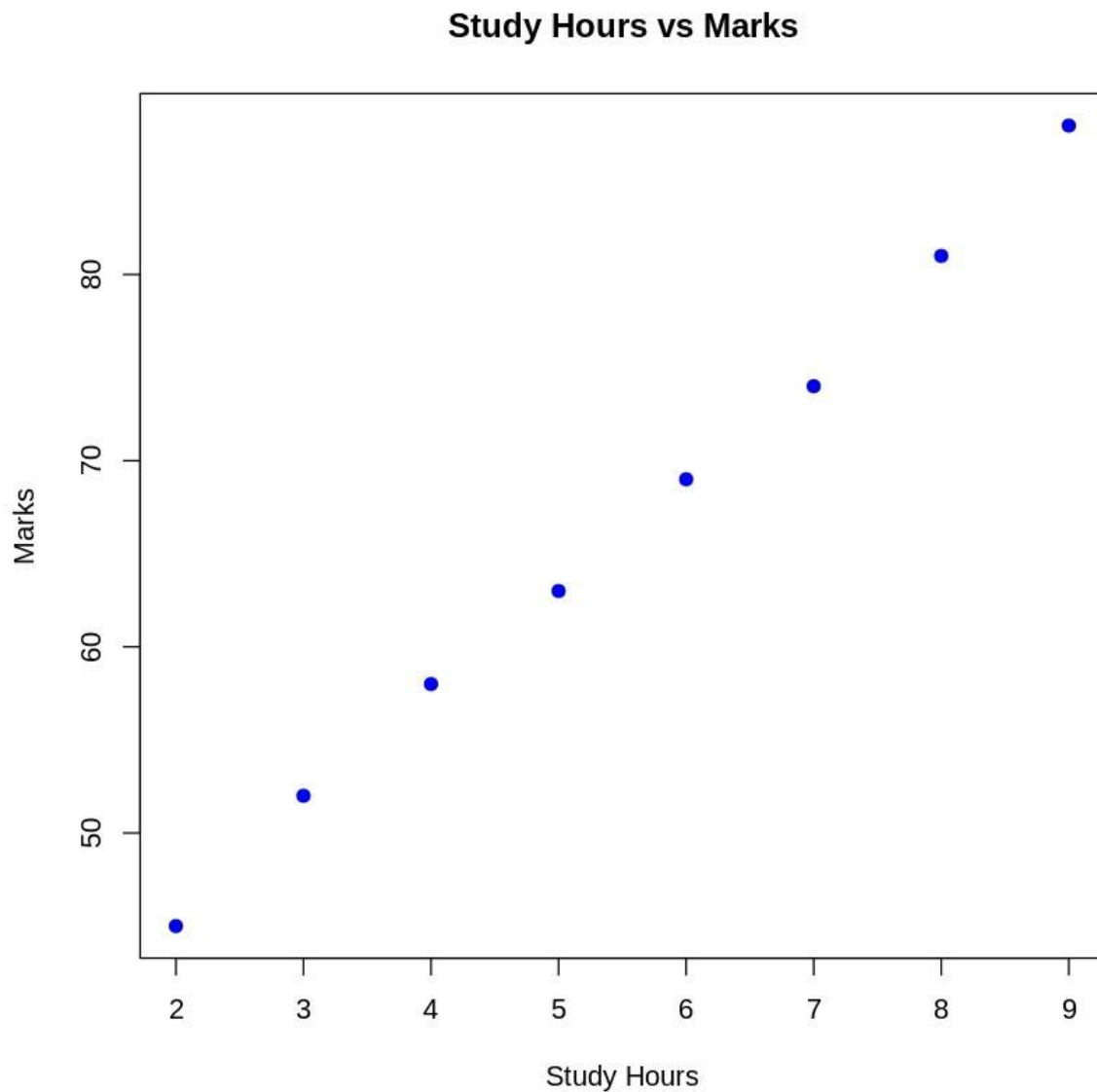
```
xlab="Study Hours",
```

```
ylab="Marks",    pch=19,
```

```
col="blue")
```

Interpretation:

The scatterplot shows a **strong positive correlation**. As study hours increase, students' marks also increase.



2. Age vs Blood Pressure (Python)

```
import matplotlib.pyplot as plt
```

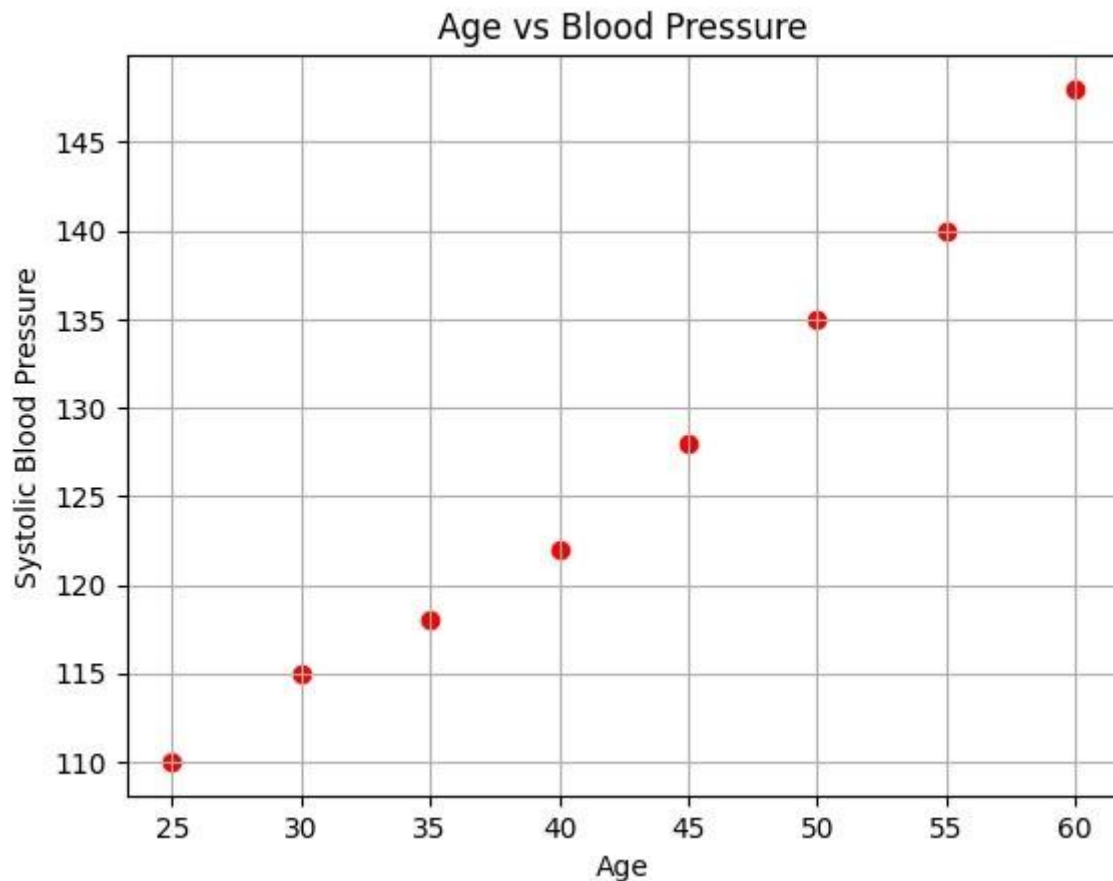
```
age = [25,30,35,40,45,50,55,60] bp =  
[110,115,118,122,128,135,140,145]
```

```
plt.scatter(age, bp, color='red') plt.xlabel("Age")  
plt.ylabel("Systolic Blood Pressure")  
plt.title("Age vs Blood Pressure") plt.grid(True)
```

```
plt.show()
```

Comment:

The scatterplot indicates a **positive relationship** between age and blood pressure. Blood pressure generally increases with age.



3. Advertising Expenditure vs Monthly Sales (Python)

```
import matplotlib.pyplot as plt
```

```
advertising = [10,20,30,40,50,60,70,80] sales  
= [100,150,190,240,300,350,410,470]
```

```
plt.scatter(advertising, sales, color='green')
```

```
plt.xlabel("Advertising Expenditure")
```

```
plt.ylabel("Monthly Sales")
```

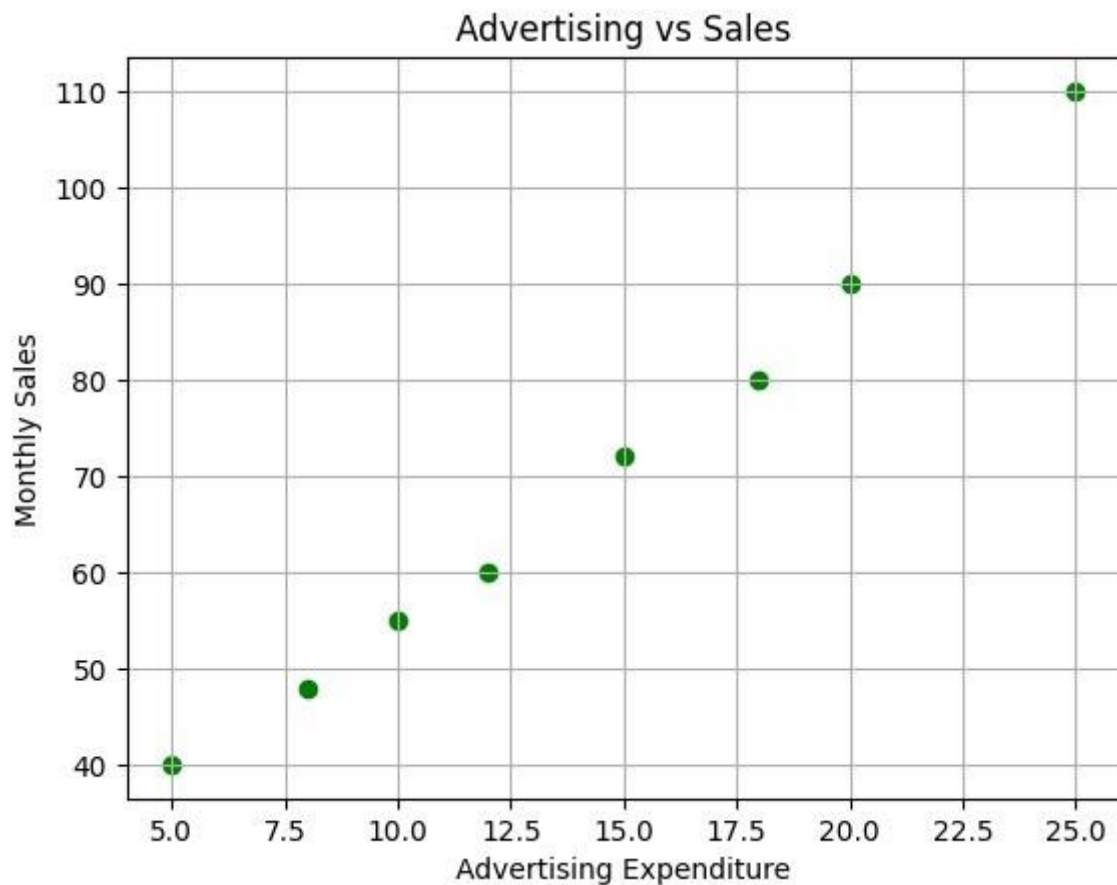
```
plt.title("Advertising vs Sales")
```

```
plt.grid(True)
```

```
plt.show()
```

Interpretation:

Monthly sales increase as advertising expenditure increases, showing a **positive correlation**.



4. Correlation Matrix and Correlogram

```
import pandas as pd
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {
    'Income': [30, 40, 50, 60, 70],
    'Savings': [5, 8, 10, 15, 20],
    'LoanAmount': [15, 20, 25, 30, 35],
    'CreditScore': [600, 650, 700, 750, 800]
}
```

```
df = pd.DataFrame(data)
```

```
corr = df.corr()
```

```
print(corr)
```

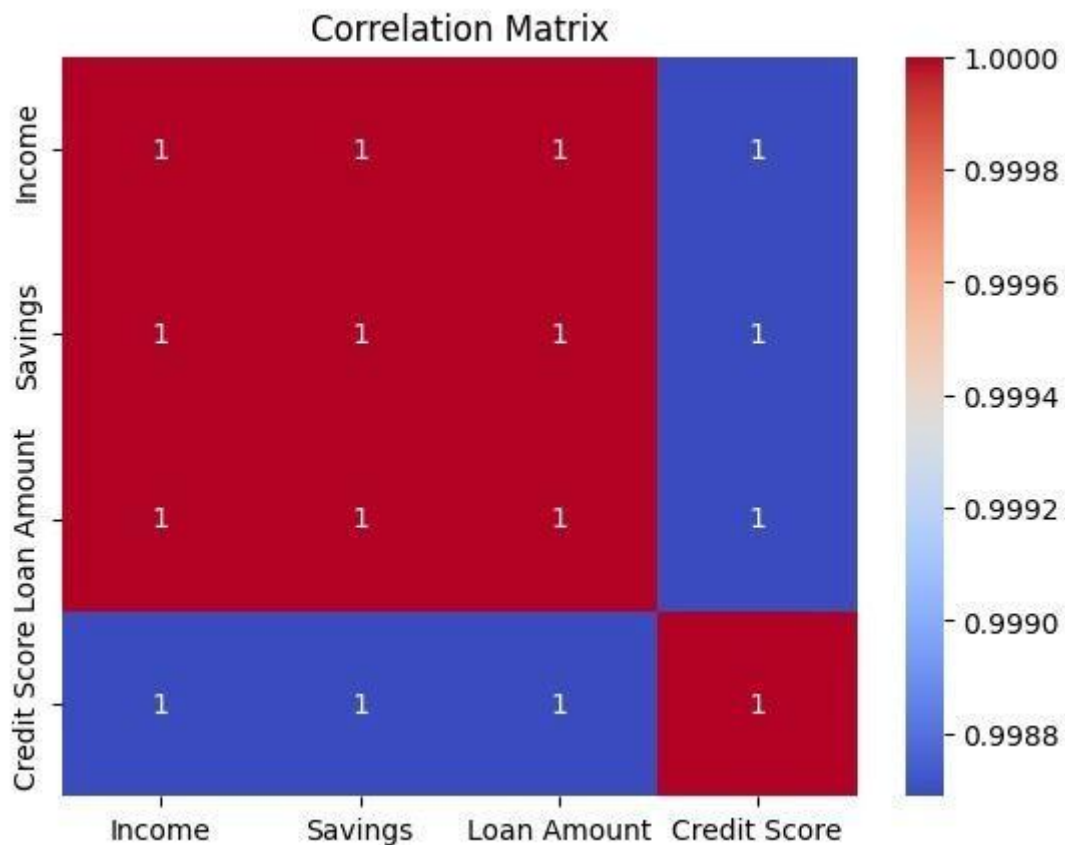
```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
plt.title("Correlation Matrix")
```

```
plt.show()
```

Answer:

The strongest positive correlation is generally between **Income and Savings**.



5. Iris Dataset Correlation

Heatmap import seaborn as sns

import matplotlib.pyplot as plt

```
iris = sns.load_dataset("iris")
```

```
corr = iris.drop("species", axis=1).corr()
```

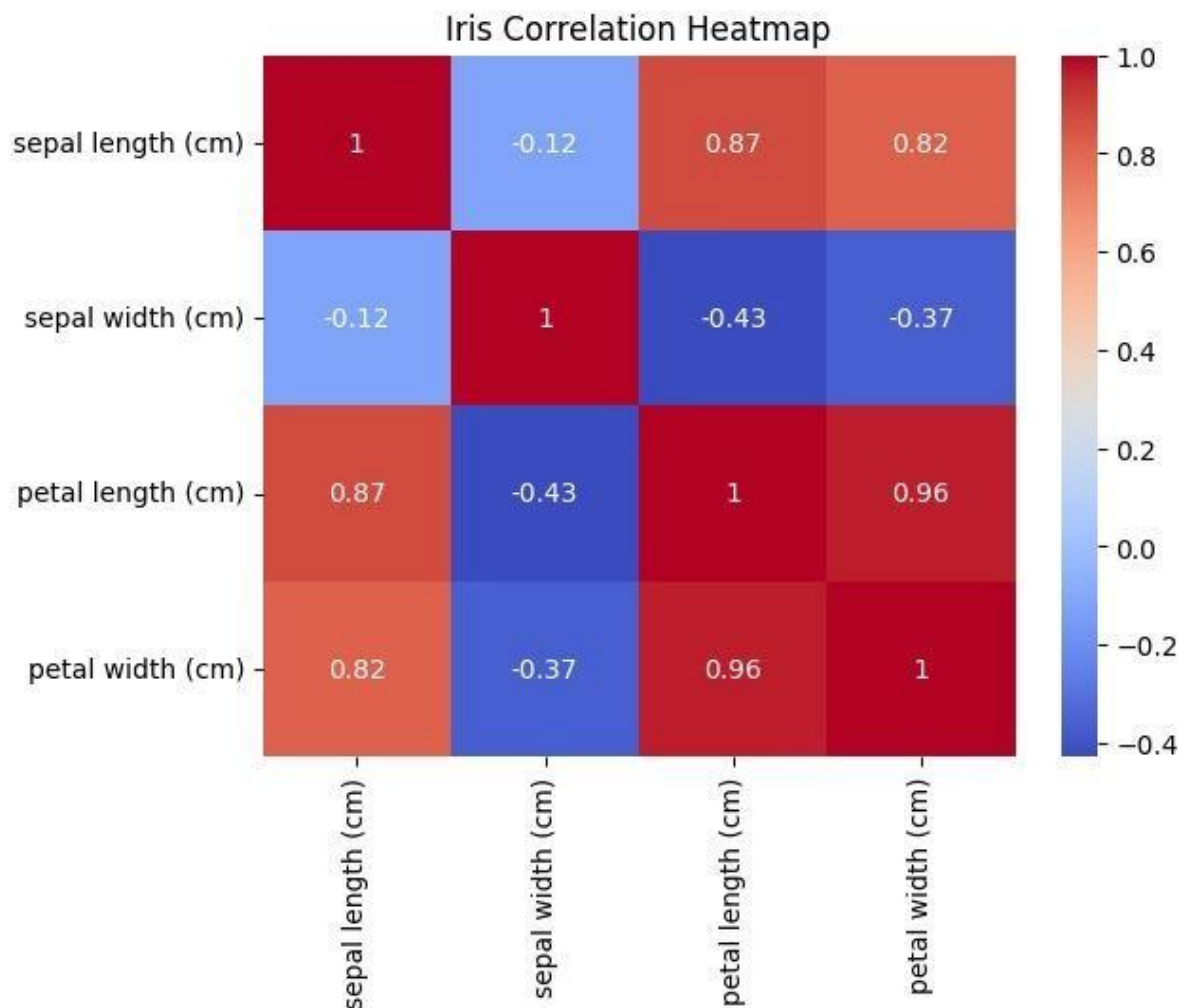
```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
plt.title("Iris Correlation Matrix")
```

```
plt.show()
```

Answer:

Negative correlation exists mainly between **Sepal Width** and **Petal Length**, and between **Sepal Width** and **Petal Width**.



6. Healthcare Correlogram

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {  
    'BMI': [22, 24, 26, 28, 30],  
    'Cholesterol': [170, 185, 195, 210, 225],  
    'BloodPressure': [110, 118, 124, 132, 140],  
    'Glucose': [80, 90, 100, 110, 120],  
}
```



```
'HeartRate':[70,72,74,76,78]
}
```

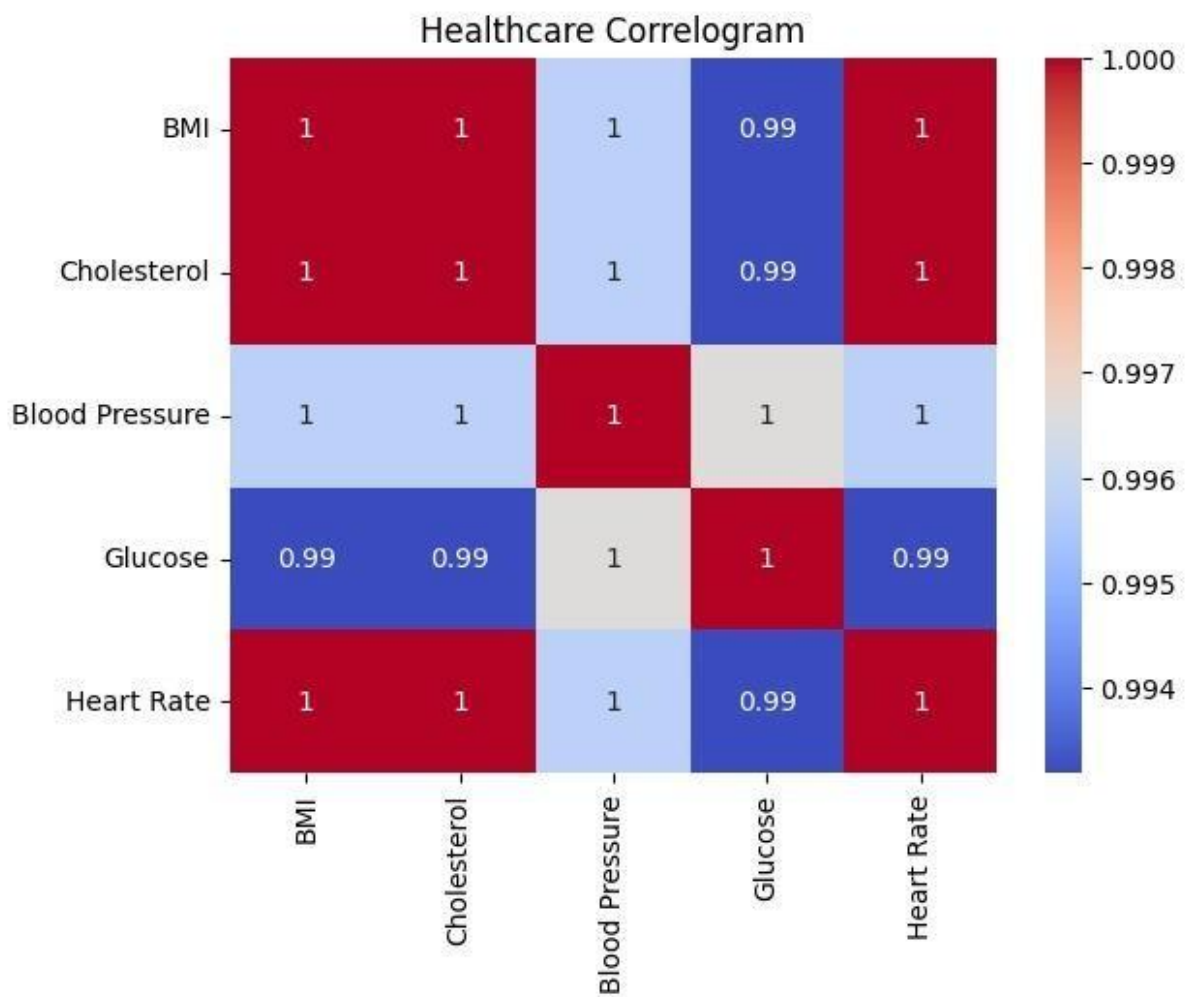
```
df = pd.DataFrame(data)
```

```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
```

```
plt.show()
```

Answer:

BMI, Cholesterol, Blood Pressure and Glucose show strong positive correlations, indicating possible multicollinearity.



7. PCA on MRI Features from

```
sklearn.decomposition import PCA from  
sklearn.datasets import make_classification  
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=100,  
                           n_features=25,  
                           random_state=42)
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

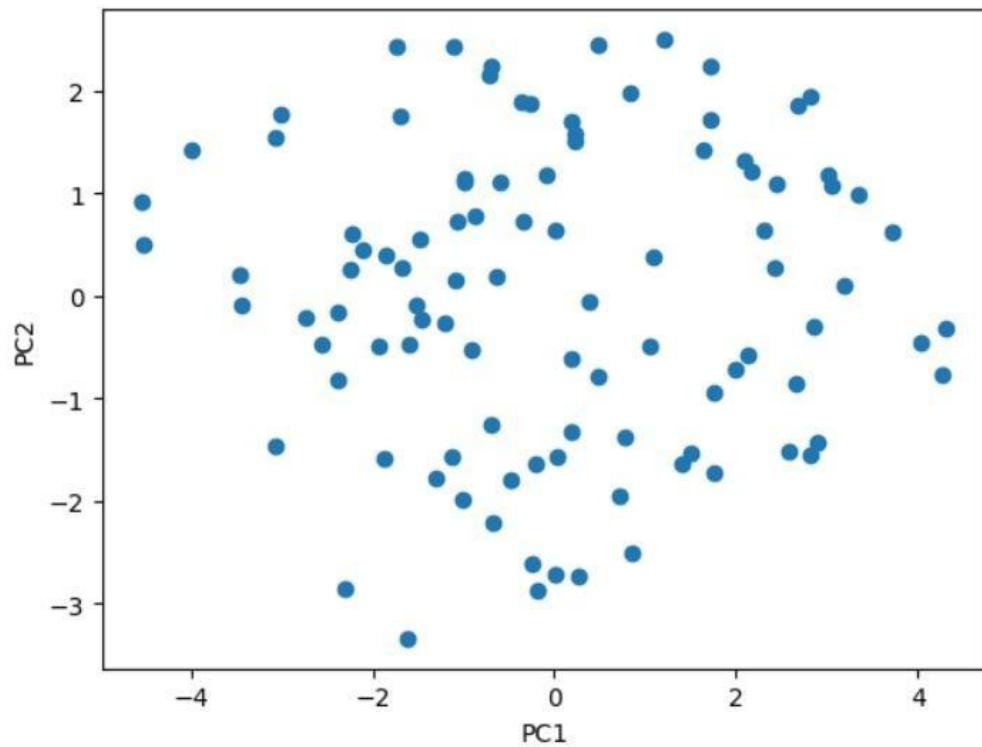
```
plt.scatter(X_pca[:,0], X_pca[:,1])
```

```
plt.xlabel("PC1") plt.ylabel("PC2")
```

```
plt.show()
```

Answer:

PCA successfully reduces 25 features into 2 principal components while preserving most of the important information.



8. PCA on Iris Dataset from

`sklearn.datasets import load_iris` from

`sklearn.decomposition import PCA`

`import matplotlib.pyplot as plt`

`iris = load_iris()`

`X = iris.data`

`pca = PCA()`

`X_pca = pca.fit_transform(X)`

`print(pca.explained_variance_ratio_)`

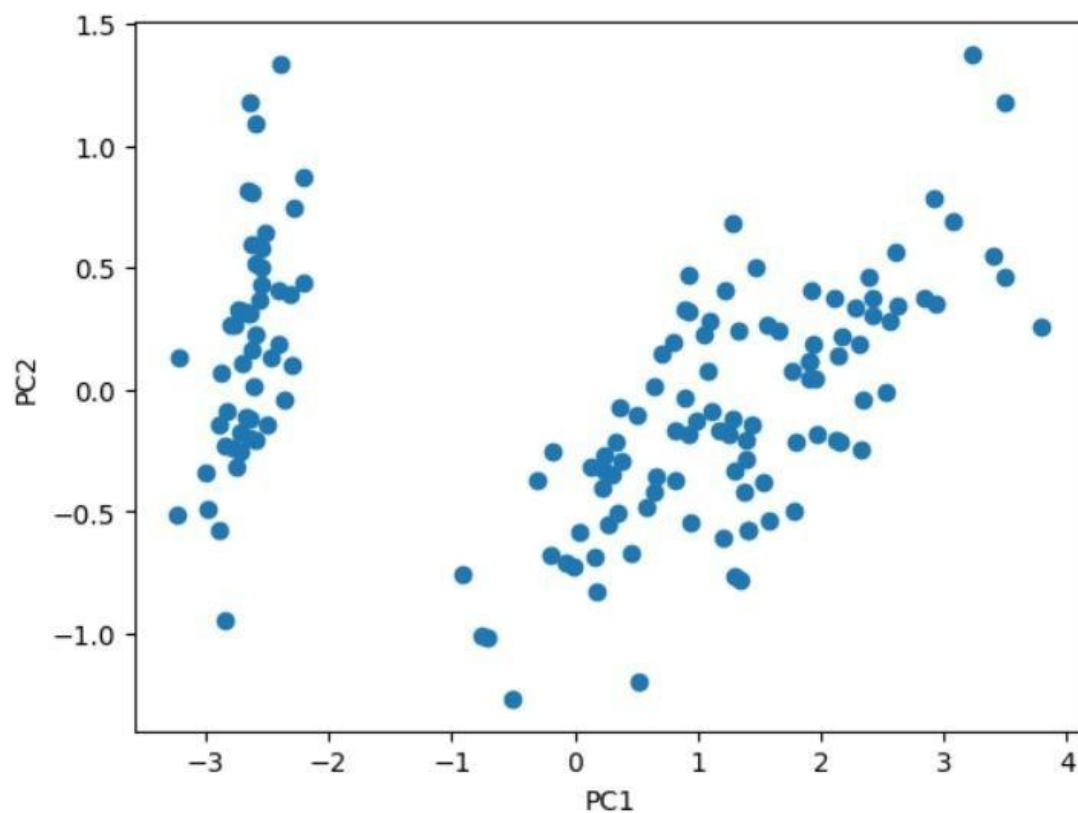
```
plt.scatter(X_pca[:,0], X_pca[:,1])
```

```
plt.xlabel("PC1") plt.ylabel("PC2")
```

```
plt.show()
```

Answer:

The **first principal component (PC1)** explains the highest variance in the Iris dataset.



9. PCA on Automobile Dataset from

```
sklearn.datasets import make_classification
```

```
from sklearn.decomposition import PCA import
```

```
matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=100,
```

```
n_features=15,  
random_state=0)
```

```
pca = PCA(n_components=2)
```

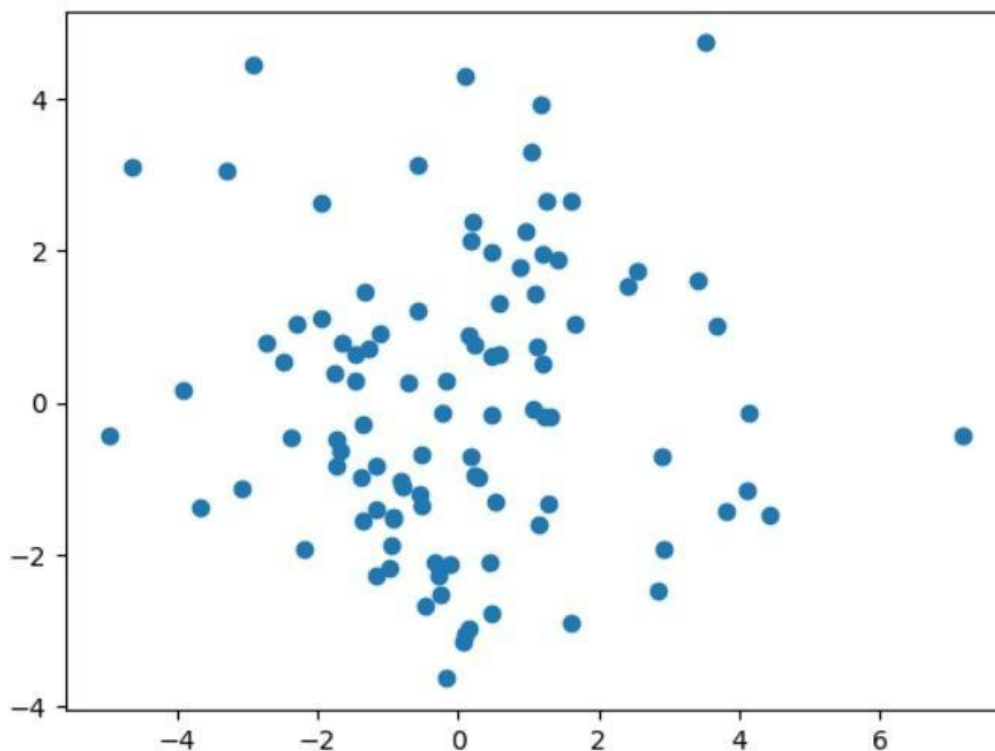
```
result = pca.fit_transform(X)
```

```
plt.scatter(result[:,0], result[:,1])
```

```
plt.show()
```

Answer:

PCA reduces the 15 specifications into two principal components for easier visualization.



10. t-SNE on Facial Recognition Dataset

from sklearn.manifold import TSNE from

```
sklearn.datasets import make_classification
import matplotlib.pyplot as plt

X, y = make_classification(n_samples=200,
                           n_features=128,
                           random_state=42)

tsne = TSNE(n_components=2, random_state=42)

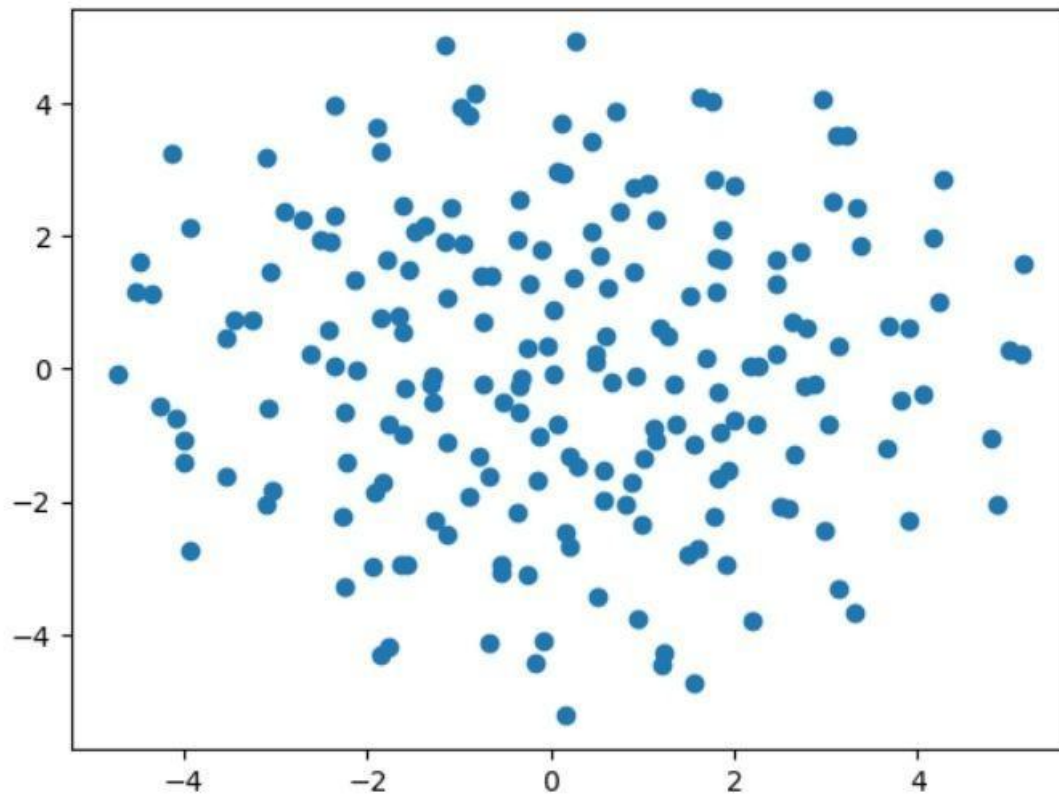
X_tsne = tsne.fit_transform(X)

plt.scatter(X_tsne[:,0], X_tsne[:,1])

plt.show()
```

Answer:

t-SNE preserves local neighborhood relationships better than PCA, making it more suitable for visualizing high-dimensional facial embeddings.



11. t-SNE on Iris Dataset from
sklearn.datasets import load_iris from
sklearn.manifold import TSNE import
matplotlib.pyplot as plt

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

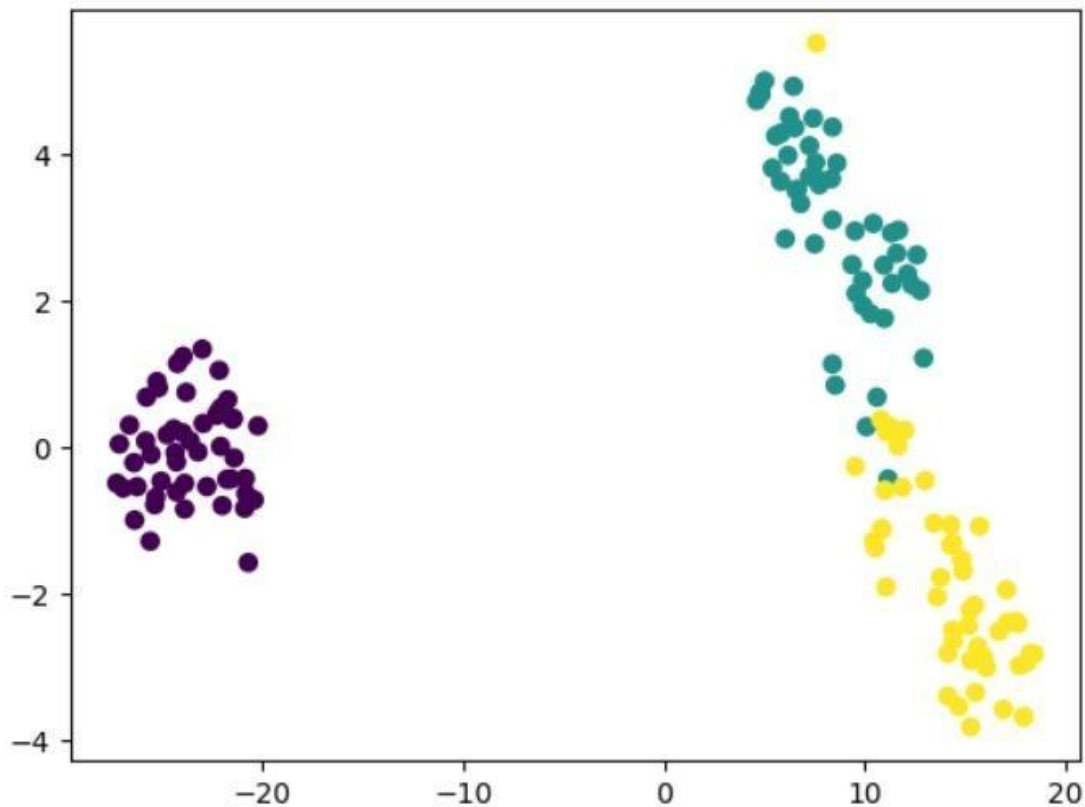
```
X_tsne = TSNE(n_components=2,  
               random_state=42).fit_transform(X)
```

```
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
```

```
plt.show()
```

Answer:

The three Iris species form distinct clusters, showing that t-SNE separates the classes effectively.



12. Customer Segmentation using t-SNE from

```
sklearn.datasets import make_classification  
from sklearn.manifold import TSNE import  
matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=300,  
                           n_features=40,  
                           random_state=42) X_tsne =  
TSNE(n_components=2,
```



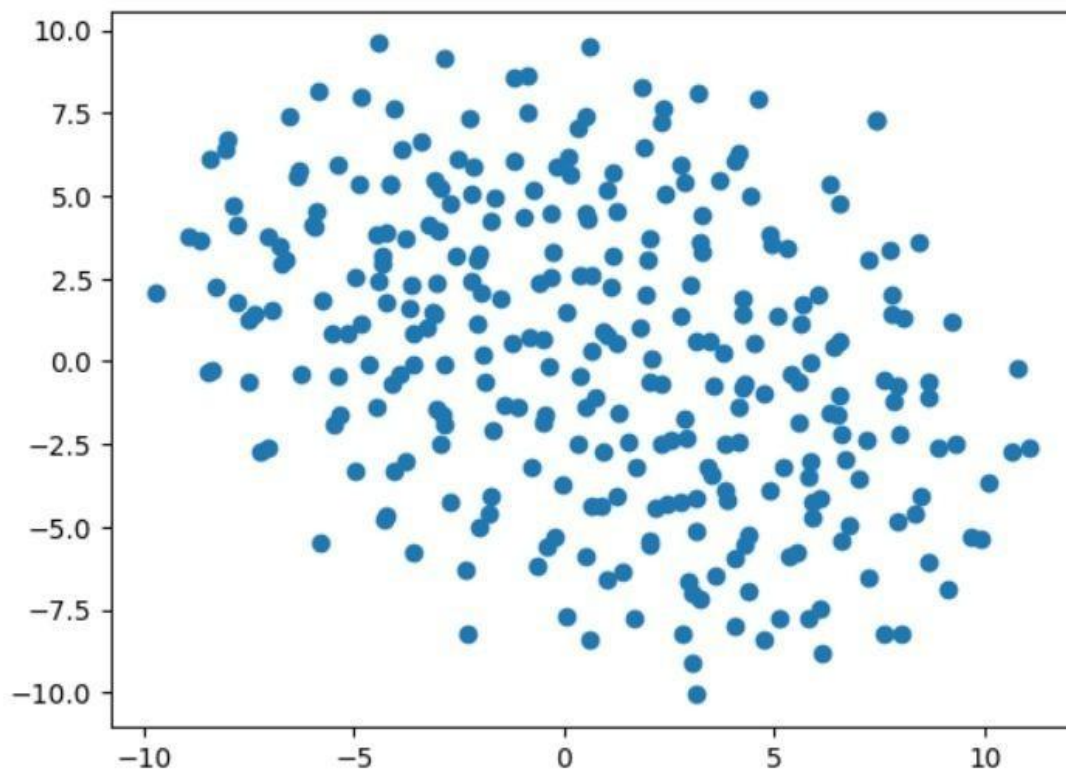
```
random_state=42).fit_transform(X)
```

```
plt.scatter(X_tsne[:,0], X_tsne[:,1])
```

```
plt.show()
```

Answer:

Distinct customer groups are visible, indicating effective customer segmentation.



13. UMAP on Medical Image Dataset

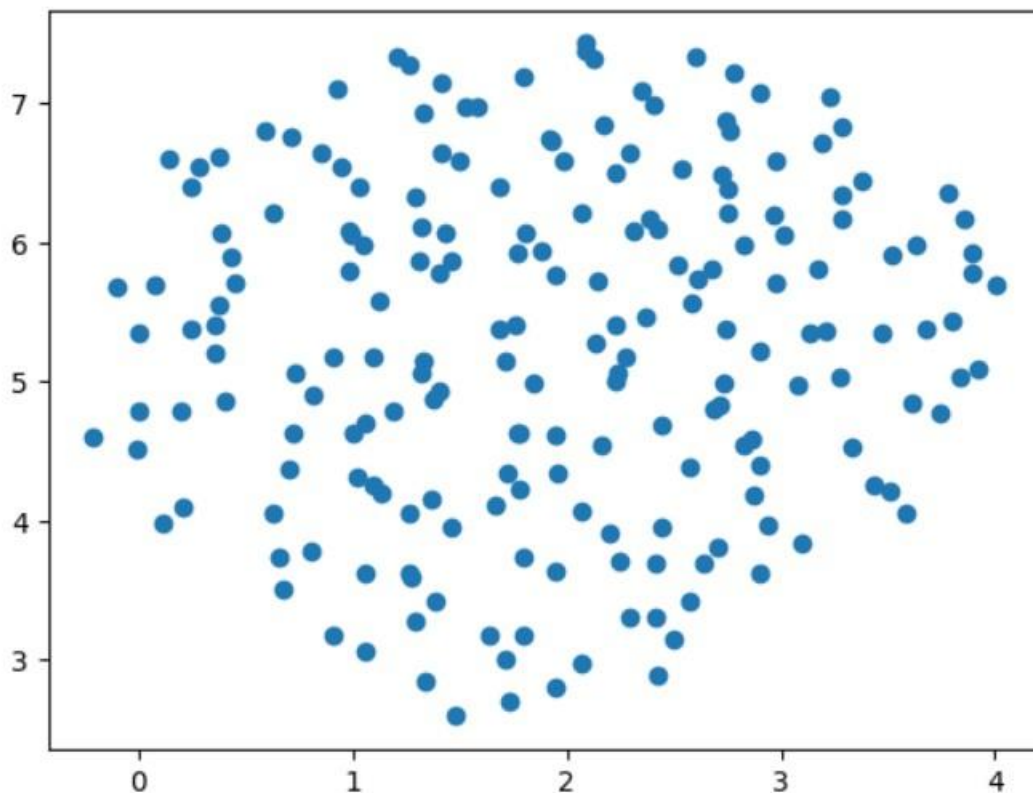
```
import umap from sklearn.datasets import  
make_classification import matplotlib.pyplot  
as plt
```

```
X, y = make_classification(n_samples=200,
```

```
n_features=100,  
random_state=42)  
  
embedding = umap.UMAP().fit_transform(X)  
  
plt.scatter(embedding[:,0], embedding[:,1])  
  
plt.show()
```

Answer:

UMAP preserves both local and global structure better than PCA, resulting in clearer clusters.



14. UMAP on Iris Dataset

```
import umap from sklearn.datasets
import load_iris import
matplotlib.pyplot as plt

iris = load_iris()

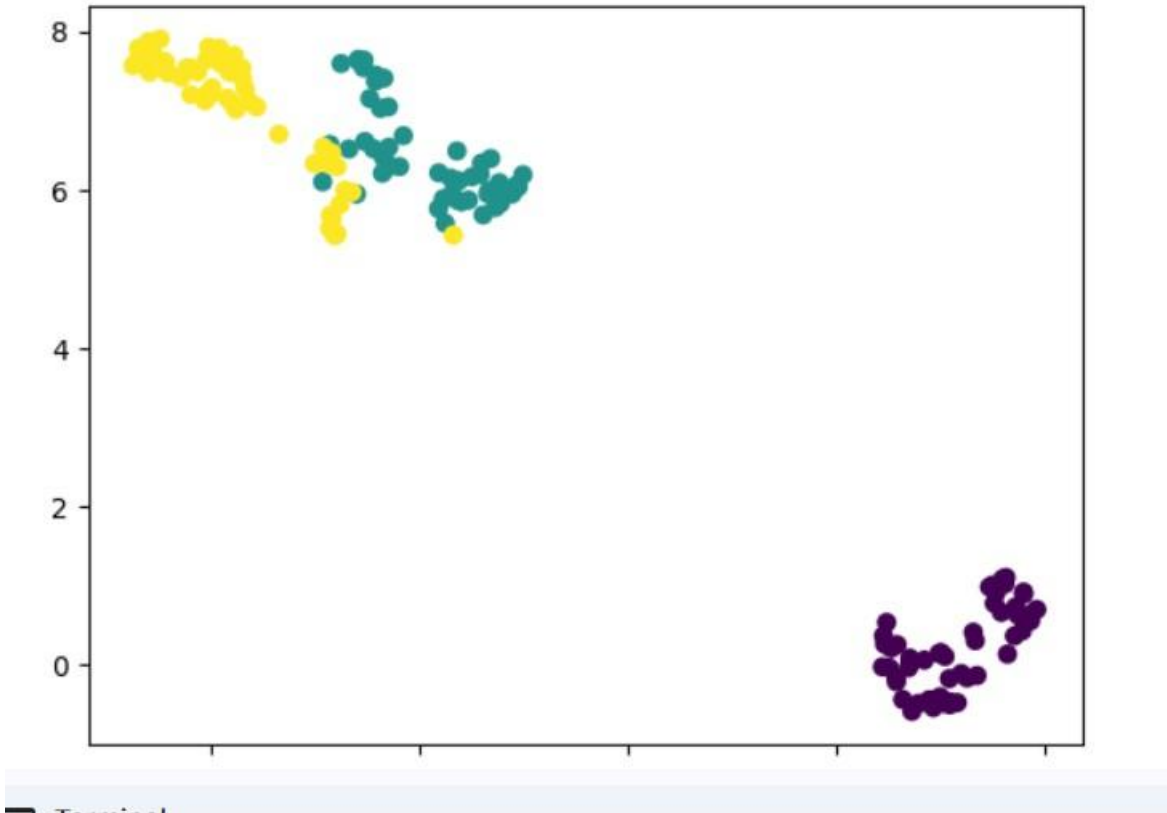
embedding = umap.UMAP(random_state=42).fit_transform(iris.data)

plt.scatter(embedding[:,0], embedding[:,1],
            c=iris.target)

plt.show()
```

Answer:

The Iris species appear as three well-separated clusters after UMAP dimensionality reduction.



15. Customer Purchasing Behaviour using

UMAP import umap from sklearn.datasets import
make_classification import matplotlib.pyplot as plt

```
X, y = make_classification(n_samples=300,  
                           n_features=60,  
                           random_state=42)
```

```
embedding = umap.UMAP().fit_transform(X)
```

```
plt.scatter(embedding[:,0], embedding[:,1])
```

```
plt.show()
```

Answer:

UMAP effectively reduces the 60-dimensional customer purchasing data into two dimensions, allowing customer clusters to be identified easily.

